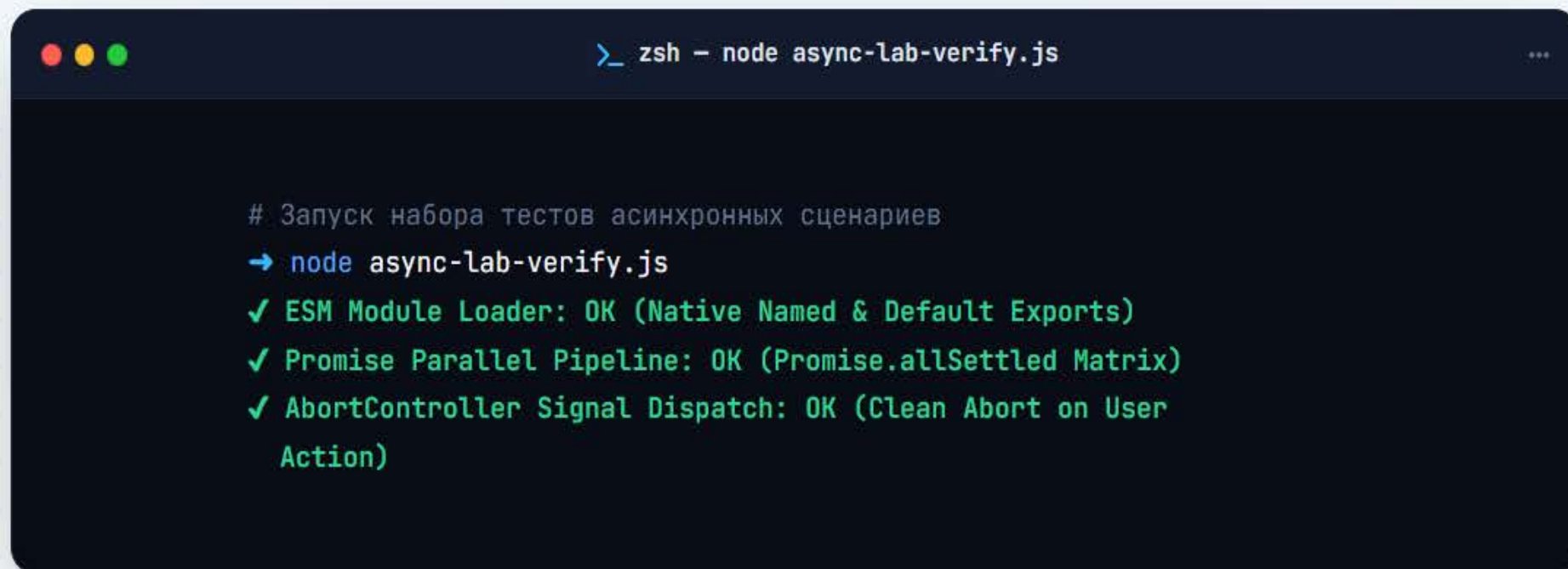


Практика: **Функции, модули, Promise и async/await**

Отработка каррирования, модульного разделения на ES-модули, параллельных комбинаторов промисов, надежных async/await сервисов с обработкой ошибок и отменой запросов.

A terminal window with a dark background and light text. The title bar shows a blue prompt character followed by 'zsh - node async-lab-verify.js'. The terminal content shows a comment in Russian, a command to run a JavaScript file, and three green checkmarks indicating successful test results for ESM Module Loader, Promise Parallel Pipeline, and AbortController Signal Dispatch.

```
> zsh - node async-lab-verify.js

# Запуск набора тестов асинхронных сценариев
→ node async-lab-verify.js
✓ ESM Module Loader: OK (Native Named & Default Exports)
✓ Promise Parallel Pipeline: OK (Promise.allSettled Matrix)
✓ AbortController Signal Dispatch: OK (Clean Abort on User Action)
```

01

Этап 1: Подготовка изолированного стенда с нативным ESM

Инициализация `package.json` с флагом `"type": "module"` для поддержки `import/export`

Чтобы запускать нативный синтаксис `import` и `export` в Node.js без сборщиков, необходимо объявить пакет как ESM-модуль.

+ 1.1. Создание структуры файлов

```
> Terminal - zsh

→ mkdir async-react-lab &&
cd async-react-lab
→ npm init
-y → npm pkg
set type="module"
# Включаем нативный синтаксис import/export
→ touch api.js mathUtils.js main.js
```

```
// 2. Каррированная фабрика проверки диапазона чисел:  
export const  
validateRange = (min, max) ⇒ value  
⇒ { return value ≥ min &&  
value ≤ max ? { valid: true } : {  
valid: false, error:  
`Число должно быть в диапазоне от ${min} до ${max}`  
}; };
```

```
// Создаем специализированные валидаторы:  
export const isPasswordSecure =  
validateMinLength(8); export const isAgeValid =  
validateRange(18, 65);
```

03

Этап 3: Создание Promise вручную с эмуляцией задержек сети

Генерация асинхронных таймеров и обработка успеха/отказа через Promise Constructor

Создадим собственный модуль задержки и генератор сетевых ответов с настраиваемым процентом вероятности ошибки.

3.1. Создание сервиса мок-запросов (api.js)

```
api.js — Mock Network Service

// Утилита задержки на базе Promise:
export const
delay = ms =>
new
Promise(resolve =>
setTimeout(resolve, ms));

// Мок-функция сетевого запроса с эмуляцией сбоев (Failure
Rate):
```

```
export async function
mockFetchUser(userId, { shouldFail =
false, delayMs =
500 } = {}) {
  await
  delay(delayMs);

  if (shouldFail) {
    throw new
    Error(`[500 Server Error] Не удалось загрузить пользователя ID:
      ${userId}`); }

  return { id: userId, name:
    `Пользователь #${userId}`, role: userId
    === "u1" ?
    "admin" :
    "developer", loadedAt:
    new Date().toLocaleTimeString() }; }

export default mockFetchUser;
```

04

Этап 4: Параллельное исполнение: Promise.all vs Promise.allSettled

Сравнение стратегий Fail-Fast и отказоустойчивой загрузки дашбордов

Отработаем параллельную загрузку независимых данных и правильную обработку частичных ошибок.

4.1. Параллельный сбор данных (main.js)

```
import { mockFetchUser }  
from  
"./api.js";  
  
// СЦЕНАРИЙ 1: Отказоустойчивая загрузка виджетов дашборда через  
Promise.allSettled  
export function
```

```
async function
loadDashboardWidgets() {
  const userIds = ["u1", "u2_failed",
    "u3"];

  const requests = userIds.map(id => mockFetchUser(id, { shouldFail:
    id.includes("failed") }) );

  const results =
    await Promise.allSettled(requests);

  const successfulUsers = [];
  const failedErrors = [];
  results.forEach((result, index) => {
    if (result.status ===
      "fulfilled") { successfulUsers.push(result.value); } else {
      failedErrors.push({ id: userIds[index],
        reason: result.reason.message }); } }); console.log("Успешно загружены:", successfulUsers);
    console.log("Ошибки загрузки виджетов:", failedErrors); }

  await
    loadDashboardWidgets();
}
```

05

Этап 5: Асинхронный клиент с отменой запроса через AbortController

Предотвращение утечек памяти и состояния гонки (Race Condition)

Создадим функцию с возможностью прерывания по сигналу или по таймауту.

5.1. Реализация сервиса с поддержкой AbortSignal

```
import { delay }
from
"./api.js";

// Асинхронная операция с поддержкой отмены по AbortSignal:
export async function
fetchDataWithCancel(url, { signal } = {})
{
  // Проверяем, не был ли сигнал уже отменен до старта:
  if (signal?.aborted) {
```

```
if (signal?.aborted) {  
  throw new  
    DOMException("Операция отменена до старта", "AbortError"); }  
  
return new  
  Promise((resolve, reject) => {  
    const timer =  
      setTimeout(() => { resolve({ data:  
        `Ответ для ${url}`, timestamp: Date.now() }); }, 1500);  
  
    // Подписываемся на событие отмены:  
    signal?.addEventListener("abort", () => { clearTimeout(timer);  
      reject(new  
        DOMException("Запрос отменен пользователем", "AbortError")); }); }); };
```